

Bike Store Sales Analysis Report

Analyst: Zyad Ammar

Project Overview

This project performs a deep-dive analysis into the **BikeStores** database to extract strategic business insights. Using advanced SQL techniques, the analysis focuses on identifying sales trends, understanding customer demographics, and evaluating product performance to support data-driven decision-making.

Data Methodology & Scope

- **Status Filtering:** To ensure accuracy, the analysis is primarily filtered for "Completed" orders (Status 4). This provides a realistic view of actual revenue and fulfilled demand.
- **Customer Behavior Note:** In the current dataset, all customers with completed orders have made exactly one purchase. Consequently, the "Top Customers" and "One-time Buyers" lists are currently identical.
- **Scalability:** The SQL queries are designed to be dynamic; they are built to automatically track and categorize repeat customers as the database updates over time, ensuring the analysis remains robust for future data growth.
-

Executive Summary of Insights

1. Sales Performance and Revenue Streams

- **Validated Revenue:** The store generated approximately **\$7.6 million** in actual revenue from completed transactions.
- **Operational Excellence:** The store achieved a high fulfillment rate of **89.5%**, with **1,445** successfully completed orders out of **1,615** total transactions. This indicates a robust supply chain and efficient order processing.
- **Category Dominance:** Revenue is heavily concentrated in specific categories. **Mountain Bikes** are the primary revenue driver, contributing **37.3%** of total sales, followed by Road Bikes at **19.9%**.

2. Customer Behavior and Retention

- **One-Time Buyers:** A significant portion of the customer base consists of one-time purchasers. This indicates a high potential for growth through improved customer

retention and loyalty strategies.

- **Target Audience:** By focusing on completed orders, we have identified the true high-value customers who form the core of the store's successful transactions.

3. Inventory and Product Optimization

- **Stagnant Inventory:** Several products in the catalog have generated zero revenue even during peak periods. This suggests a need for inventory re-evaluation or aggressive marketing for slow-moving items.
- **Strategic Discounting:** Analysis shows that while discounts drive volume, their impact varies across categories, suggesting room for price optimization to protect profit margins.

Strategic Recommendations

1. **Retention Campaigns:** Launch targeted marketing to convert one-time buyers into repeat customers through personalized offers and loyalty rewards.
2. **Focus on Winners:** Reallocate marketing budget and stock levels toward high-growth categories (Mountain and Road bikes).
3. **Inventory Liquidation:** Implement clearance strategies for products with no sales history to improve cash flow and warehouse efficiency.

Data Extraction & Technical Implementation

In this section, we translate the business requirements into **SQL queries**. The analysis is divided into key modules to systematically explore the database and validate the insights presented in the executive summary.

Note: All queries below are executed on the **BikeStores** database using **Microsoft SQL Server**.

1. Sales Overview

Total Revenue, Orders, Customers :

```
In [19]: %%sql
SELECT
    COUNT(DISTINCT c.customer_id) AS total_customers,
```

```

COUNT(DISTINCT o.order_id) AS total_orders,
ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS total_revenue
FROM sales.customers c
JOIN sales.orders o ON c.customer_id = o.customer_id
JOIN sales.order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 4;

```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes

Done.

Out[19]: **total_customers total_orders total_revenue**

total_customers	total_orders	total_revenue
1445	1445	6662615.0000

Average Order Value :

In [23]: **%%sql**

```

SELECT
    ROUND(
        SUM(oi.quantity * oi.list_price * (1 - oi.discount))
        / COUNT(DISTINCT o.order_id), 0
    ) AS avg_order_value
FROM sales.orders o
JOIN sales.order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 4;

```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes

Done.

Out[23]: **avg_order_value**

avg_order_value
4611.000000

Total Quantity Sold :

In [25]: **%%sql**

```

SELECT
    SUM(oi.quantity) AS total_quantity
FROM sales.order_items oi
JOIN sales.orders o ON o.order_id = oi.order_id
WHERE o.order_status = 4;

```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes

Done.

Out[25]: **total_quantity**

total_quantity
6318

2. Store Analysis

Revenue by Store :

```
In [26]: %%sql
SELECT
    s.store_name,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS total_revenue
FROM sales.stores s
JOIN sales.orders o ON s.store_id = o.store_id
JOIN sales.order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 4
GROUP BY s.store_name
ORDER BY total_revenue DESC;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

```
Out[26]:
```

store_name	total_revenue
Baldwin Bikes	4701210.0000
Santa Cruz Bikes	1255492.0000
Rowlett Bikes	705914.0000

Store Performance :

```
In [30]: %%sql
SELECT
    s.store_name,
    COUNT(DISTINCT o.order_id) AS total_orders,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS total_revenue,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)) * 1.0
        / COUNT(DISTINCT o.order_id),0) AS revenue_per_order
FROM sales.stores s
JOIN sales.orders o
    ON s.store_id = o.store_id AND o.order_status = 4
JOIN sales.order_items oi
    ON o.order_id = oi.order_id
GROUP BY s.store_name;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

```
Out[30]:
```

store_name	total_orders	total_revenue	revenue_per_order
Baldwin Bikes	1019	4701210.0000	4614.000000
Santa Cruz Bikes	284	1255492.0000	4421.000000
Rowlett Bikes	142	705914.0000	4971.000000

3. Product and Category Analysis

Revenue by Category :

```
In [32]: %%sql
SELECT
    c.category_name,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS total_revenue
FROM sales.order_items oi
JOIN sales.orders o ON o.order_id = oi.order_id
JOIN production.products p ON p.product_id = oi.product_id
JOIN production.categories c ON c.category_id = p.category_id
WHERE o.order_status = 4
GROUP BY c.category_name
ORDER BY total_revenue DESC;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

```
Out[32]:
```

category_name	total_revenue
Mountain Bikes	2486420.0000
Road Bikes	1327156.0000
Cruisers Bicycles	866524.0000
Electric Bikes	733494.0000
Cyclocross Bicycles	642585.0000
Comfort Bicycles	346450.0000
Children Bicycles	259986.0000

Revenue by Brand :

```
In [34]: %%sql
SELECT
    b.brand_name,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS total_revenue
FROM sales.order_items oi
JOIN sales.orders o ON o.order_id = oi.order_id
JOIN production.products p ON p.product_id = oi.product_id
JOIN production.brands b ON b.brand_id = p.brand_id
WHERE o.order_status = 4
GROUP BY b.brand_name
ORDER BY total_revenue DESC;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[34]: **brand_name** **total_revenue**

Trek	3919363.0000
Electra	1019611.0000
Surly	868807.0000
Sun Bicycles	310418.0000
Haro	163783.0000
Heller	157220.0000
Pure Cycles	143214.0000
Ritchey	78186.0000
Strider	2014.0000

Top 5 Products :

```
In [35]: %%sql
SELECT *
FROM (
    SELECT
        p.product_name,
        ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS total_reve
        DENSE_RANK() OVER (ORDER BY SUM(oi.quantity * oi.list_price * (1 - oi.disco
    FROM sales.order_items oi
    JOIN sales.orders o ON o.order_id = oi.order_id
    JOIN production.products p ON p.product_id = oi.product_id
    WHERE o.order_status = 4
    GROUP BY p.product_name
) t
WHERE rn <= 5;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&t
rusted_connection=yes
Done.

Out[35]: **product_name** **total_revenue** **m**

Trek Slash 8 27.5 - 2016	544319.0000	1
Trek Conduit+ - 2016	381149.0000	2
Trek Fuel EX 8 29 - 2016	354814.0000	3
Surly Straggler 650b - 2016	220714.0000	4
Trek Remedy 29 Carbon Frameset - 2016	199997.0000	5

Unsold Products :

```
In [37]: %%sql
SELECT
    p.product_id,
```

```
    p.product_name
FROM production.products p
WHERE NOT EXISTS (
    SELECT 1
    FROM sales.order_items oi
    JOIN sales.orders o
        ON o.order_id = oi.order_id
    WHERE oi.product_id = p.product_id
        AND o.order_status = 4
);
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[37]:	product_id	product_name
	284	Electra Savannah 1 (20-inch) - Girl's - 2018
	315	Electra Townie Original 21D Ladies' - 2018
	158	Trek CrossRip 1 - 2018
	195	Electra Townie Go! 8i Ladies' - 2018
	146	Trek Domane SLR 6 - 2018
	212	Electra Townie Original 21D - 2018
	218	Electra Cruiser 7D - 2016/2017/2018
	318	Trek Checkpoint ALR 5 Women's - 2019
	321	Trek Checkpoint ALR Frameset - 2019
	259	Electra Amsterdam Royal 8i Ladies - 2018
	267	Trek Precaliber 12 Girl's - 2018
	121	Surly Krampus Frameset - 2018
	319	Trek Checkpoint SL 5 Women's - 2019
	156	Trek Domane SL Frameset - 2018
	256	Electra Townie Commute 27D - 2018
	293	Electra Tiger Shark 1 (20-inch) - Boys' - 2018
	1	Trek 820 - 2016
	279	Trek Precaliber 24 7-speed Girl's - 2018
	316	Trek Checkpoint ALR 4 Women's - 2019
	176	Trek Madone 9 Frameset - 2018
	196	Trek Verve+ - 2018
	119	Trek Kids' Neko - 2018
	211	Surly Straggler 650b - 2018
	288	Electra Straight 8 1 (20-inch) - Boy's - 2018
	125	Trek Kids' Dual Sport - 2018
	317	Trek Checkpoint ALR 5 - 2019
	291	Electra Sweet Ride 1 (20-inch) - Girl's - 2018
	285	Electra Soft Serve 1 (16-inch) - Girl's - 2018
	231	Electra Cruiser Lux 3i Ladies' - 2018
	314	Electra Townie Original 21D EQ Ladies' - 2018

product_id	product_name
122	Surly Troll Frameset - 2018
202	Trek Powerfly 5 Women's - 2018
309	Electra Townie Commute 27D - 2018
280	Trek Superfly 24 - 2017/2018
177	Trek Domane SLR 6 Disc - 2018
154	Trek Domane SLR 6 Disc Women's - 2018
183	Trek Domane SL 5 Women's - 2018
303	Electra Townie Go! 8i - 2017/2018
289	Electra Superbolt 1 20" - 2018
312	Electra Townie Commute 8D Ladies' - 2018
266	Trek Superfly 20 - 2018
174	Trek Domane SLR Disc Frameset - 2018
320	Trek Checkpoint SL 6 - 2019

Category Contribution :

```
In [40]: %%sql
SELECT
    c.category_name,
    Round(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS revenue,
    ROUND(
        SUM(oi.quantity * oi.list_price * (1 - oi.discount)) * 100.0 /
        SUM(SUM(oi.quantity * oi.list_price * (1 - oi.discount))) OVER(),2
    ) AS contribution_pct
FROM sales.order_items oi
JOIN sales.orders o ON o.order_id = oi.order_id
JOIN production.products p ON p.product_id = oi.product_id
JOIN production.categories c ON c.category_id = p.category_id
WHERE o.order_status = 4
GROUP BY c.category_name;
```

```
* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&t
rusted_connection=yes
Done.
```

Out[40]:

category_name	revenue	contribution_pct
Comfort Bicycles	346450.0000	5.200000
Electric Bikes	733494.0000	11.010000
Road Bikes	1327156.0000	19.920000
Cruisers Bicycles	866524.0000	13.010000
Mountain Bikes	2486420.0000	37.320000
Children Bicycles	259986.0000	3.900000
Cyclocross Bicycles	642585.0000	9.640000

4. Customer Analysis

Orders per Customer :

In [43]:

```
%%sql
SELECT top 20
    c.customer_id,
    (c.first_name + ' ' + c.last_name) AS customer_name,
    COUNT(o.order_id) AS total_orders
FROM sales.customers c
LEFT JOIN sales.orders o
    ON c.customer_id = o.customer_id
    AND o.order_status = 4
GROUP BY c.customer_id, (c.first_name + ' ' + c.last_name);
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[43]:

customer_id	customer_name	total_orders
1174	Aaron Knapp	1
338	Abbey Pugh	1
75	Abby Gamble	1
1224	Abram Copeland	1
673	Adam Henderson	1
1085	Adam Thornton	1
195	Addie Hahn	1
1261	Adelaida Hancock	1
22	Adelle Larsen	1
1023	Adena Blake	1
1412	Adrien Hunter	1
574	Adriene Rivera	1
1252	Adriene Rollins	1
527	Afton Juarez	1
1353	Agatha Daniels	1
769	Agatha Melton	1
771	Agnes Sims	1
1181	Agustina Lawrence	1
1322	Ai Forbes	1
937	Aida Koch	1

Customers with One Order :

In [46]:

```
%%sql
SELECT top 20
    c.customer_id,
    (c.first_name + ' ' + c.last_name) AS customer_name,
    COUNT(o.order_id) AS total_orders
FROM sales.customers c
LEFT JOIN sales.orders o
    ON c.customer_id = o.customer_id
    AND o.order_status = 4
GROUP BY c.customer_id, (c.first_name + ' ' + c.last_name)
HAVING COUNT(o.order_id) = 1;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[46]:

customer_id	customer_name	total_orders
-------------	---------------	--------------

1174	Aaron Knapp	1
338	Abbey Pugh	1
75	Abby Gamble	1
1224	Abram Copeland	1
673	Adam Henderson	1
1085	Adam Thornton	1
195	Addie Hahn	1
1261	Adelaida Hancock	1
22	Adelle Larsen	1
1023	Adena Blake	1
1412	Adrien Hunter	1
574	Adriene Rivera	1
1252	Adriene Rollins	1
527	Afton Juarez	1
1353	Agatha Daniels	1
769	Agatha Melton	1
771	Agnes Sims	1
1181	Agustina Lawrence	1
1322	Ai Forbes	1
937	Aida Koch	1

Top 10 Customers :

```
In [47]: %%sql
SELECT TOP 10
    c.customer_id,
    (c.first_name + ' ' + c.last_name) AS customer_name,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS total_spending
FROM sales.customers c
JOIN sales.orders o ON c.customer_id = o.customer_id
JOIN sales.order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 4
GROUP BY c.customer_id,(c.first_name + ' ' + c.last_name)
ORDER BY total_spending DESC;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[47]:

customer_id	customer_name	total_spending
-------------	---------------	----------------

73	Melanie Hayes	27051.0000
122	Shena Carter	24891.0000
1224	Abram Copeland	24607.0000
1214	Brigid Sharp	20649.0000
425	Augustina Joyner	20509.0000
238	Cindi Larson	20178.0000
1023	Adena Blake	19330.0000
464	Bess McBride	18853.0000
1100	Penny Acevedo	18671.0000
227	Danielle Bond	18554.0000

Customer Segmentation :

```
In [50]: %%sql
WITH CustomerOrders AS (
    SELECT
        c.customer_id,
        COUNT(o.order_id) AS order_count
    FROM sales.customers c
    LEFT JOIN sales.orders o ON c.customer_id = o.customer_id AND o.order_status = 
    GROUP BY c.customer_id
)
SELECT
    CASE
        WHEN order_count = 1 THEN 'One-time'
        WHEN order_count > 1 THEN 'Repeat'
        ELSE 'No Orders'
    END AS customer_type,
    COUNT(*) AS total_customers
FROM CustomerOrders
GROUP BY
    CASE
        WHEN order_count = 1 THEN 'One-time'
        WHEN order_count > 1 THEN 'Repeat'
        ELSE 'No Orders'
    END;
```

```
* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.
```

Out[50]:

customer_type	total_customers
---------------	-----------------

One-time	1445
----------	------

5. Order Behavior

Order Status Distribution :

```
In [51]: %%sql
SELECT
    CASE
        WHEN order_status = 1 THEN 'Pending'
        WHEN order_status = 2 THEN 'Processing'
        WHEN order_status = 3 THEN 'Rejected'
        WHEN order_status = 4 THEN 'Completed'
    END AS status_name,
    COUNT(*) AS total_orders
FROM sales.orders
GROUP BY order_status;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[51]: **status_name** **total_orders**

Rejected	45
Pending	62
Completed	1445
Processing	63

Average Items per Order :

```
In [53]: %%sql
SELECT
    ROUND(AVG(total_items * 1.0),2) AS avg_items_per_order
FROM (
    SELECT
        o.order_id,
        SUM(oi.quantity) AS total_items
    FROM sales.orders o
    JOIN sales.order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 4
    GROUP BY o.order_id
) t;
```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[53]: **avg_items_per_order**

4.370000

Orders with Maximum Items :

```
In [56]: %%sql
SELECT *
```

```

FROM (
    SELECT
        o.order_id,
        sum(quantity) as total_products,
        DENSE_RANK() over(order by sum(quantity) desc) as Rank
    FROM sales.order_items oi
    JOIN sales.orders o ON o.order_id = oi.order_id
    WHERE o.order_status = 4
    GROUP BY o.order_id ) t
WHERE Rank=1;

```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[56]:

order_id	total_products	Rank
100	10	1
518	10	1
813	10	1
944	10	1

6. Time Analysis

In [57]:

```

%%sql
SELECT
    YEAR(o.order_date) AS year,
    MONTH(o.order_date) AS month,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS revenue
FROM sales.orders o
JOIN sales.order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 4
GROUP BY YEAR(o.order_date), MONTH(o.order_date)
ORDER BY year, month;

```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[57]:

year	month	revenue
2016	1	215146.0000
2016	2	148506.0000
2016	3	180600.0000
2016	4	164399.0000
2016	5	197179.0000
2016	6	205464.0000
2016	7	199557.0000
2016	8	212827.0000
2016	9	271357.0000
2016	10	212078.0000
2016	11	176051.0000
2016	12	189685.0000
2017	1	285616.0000
2017	2	281968.0000
2017	3	308157.0000
2017	4	220409.0000
2017	5	263045.0000
2017	6	375578.0000
2017	7	213720.0000
2017	8	273337.0000
2017	9	293405.0000
2017	10	303422.0000
2017	11	268809.0000
2017	12	259506.0000
2018	1	381430.0000
2018	2	197372.0000
2018	3	363991.0000

Best Month :

In [58]:

```
%%sql
SELECT TOP 1
    YEAR(o.order_date) AS year,
```

```

    MONTH(o.order_date) AS month,
    ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS revenue
FROM sales.orders o
JOIN sales.order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = 4
GROUP BY YEAR(o.order_date), MONTH(o.order_date)
ORDER BY revenue DESC;

```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[58]:

year	month	revenue
2018	1	381430.0000

Month over Month Growth :

In [62]:

```

%%sql
WITH m AS (
    SELECT
        YEAR(order_date) AS year,
        MONTH(order_date) AS month,
        ROUND(SUM(oi.quantity * oi.list_price * (1 - oi.discount)), 0) AS revenue
    FROM sales.orders o
    JOIN sales.order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 4
    GROUP BY YEAR(order_date), MONTH(order_date)
)
SELECT
    *,
    ROUND(LAG(revenue) OVER (ORDER BY year, month), 0) AS prev_month,
    ROUND(revenue - LAG(revenue) OVER (ORDER BY year, month), 0) AS growth,
    ROUND((revenue - LAG(revenue) OVER (ORDER BY year, month)) * 100.0
    / NULLIF(LAG(revenue) OVER (ORDER BY year, month), 0), 2) AS growth_pct
FROM m;

```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[62]:

year	month	revenue	prev_month	growth	growth_pct
2016	1	215146.0000	None	None	None
2016	2	148506.0000	215146.0000	-66640.0000	-30.970000
2016	3	180600.0000	148506.0000	32094.0000	21.610000
2016	4	164399.0000	180600.0000	-16201.0000	-8.970000
2016	5	197179.0000	164399.0000	32780.0000	19.940000
2016	6	205464.0000	197179.0000	8285.0000	4.200000
2016	7	199557.0000	205464.0000	-5907.0000	-2.870000
2016	8	212827.0000	199557.0000	13270.0000	6.650000
2016	9	271357.0000	212827.0000	58530.0000	27.500000
2016	10	212078.0000	271357.0000	-59279.0000	-21.850000
2016	11	176051.0000	212078.0000	-36027.0000	-16.990000
2016	12	189685.0000	176051.0000	13634.0000	7.740000
2017	1	285616.0000	189685.0000	95931.0000	50.570000
2017	2	281968.0000	285616.0000	-3648.0000	-1.280000
2017	3	308157.0000	281968.0000	26189.0000	9.290000
2017	4	220409.0000	308157.0000	-87748.0000	-28.480000
2017	5	263045.0000	220409.0000	42636.0000	19.340000
2017	6	375578.0000	263045.0000	112533.0000	42.780000
2017	7	213720.0000	375578.0000	-161858.0000	-43.100000
2017	8	273337.0000	213720.0000	59617.0000	27.890000
2017	9	293405.0000	273337.0000	20068.0000	7.340000
2017	10	303422.0000	293405.0000	10017.0000	3.410000
2017	11	268809.0000	303422.0000	-34613.0000	-11.410000
2017	12	259506.0000	268809.0000	-9303.0000	-3.460000
2018	1	381430.0000	259506.0000	121924.0000	46.980000
2018	2	197372.0000	381430.0000	-184058.0000	-48.250000
2018	3	363991.0000	197372.0000	166619.0000	84.420000

Year over Year Growth :

In [64]:

```

%%sql
WITH y AS (
SELECT

```

```

        YEAR(order_date) AS year,
        Round(SUM(oi.quantity * oi.list_price * (1 - oi.discount)),0) AS revenue
    FROM sales.orders o
    JOIN sales.order_items oi ON o.order_id = oi.order_id
    WHERE o.order_status = 4
    GROUP BY YEAR(order_date)
)
SELECT
    *,
    ROUND(LAG(revenue) OVER (ORDER BY year), 0) AS prev_year,
    ROUND(revenue - LAG(revenue) OVER (ORDER BY year), 0) AS yoy_growth,
    ROUND((revenue - LAG(revenue) OVER (ORDER BY year)) * 100.0 / NULLIF(LAG(revenue), 0)) AS yoy_growth_pct
FROM y;

```

* mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes
Done.

Out[64]:

	year	revenue	prev_year	yoy_growth	yoy_growth_pct
	2016	2372849.0000	None	None	None
	2017	3346973.0000	2372849.0000	974124.0000	41.050000
	2018	942793.0000	3346973.0000	-2404180.0000	-71.830000

In [2]: %config SqlMagic.style = '_DEPRECATED_DEFAULT'

In [14]: %load_ext sql

The sql extension is already loaded. To reload it, use:
%reload_ext sql

In [15]: %sql mssql+pyodbc://@DESKTOP-41VDPK1/BikeStores?driver=ODBC+Driver+17+for+SQL+Serve